



Phase IV: Software Testing

CEN 302 – Software Engineering

Prepared by: Group III – Thesis Progress Tracker

Delivered to: M.Sc. Klea Luka

Table of Contents:

1. Introduction to Testing.....	3
2. Purpose of Testing.....	3
3. Focus on Testing a Single Component.....	3
4. Preparing Test Cases.....	5
Case 1: Admin adds a new user.....	5
Case 2: Milestone Tracking and Thesis Submission.....	6
5. Writing Test Code.....	7
Case 1: Admin adds a new user.....	7
Case 2: Milestone Tracking.....	12
6. Running Tests.....	16
Case 1: Admin adds a new user.....	16
Case 2: Milestone Tracking and Thesis Submission.....	16
7. Test Coverage.....	17
Introduction.....	17
Chosen Component.....	17
Test Cases.....	18
Testing Tools.....	18
Test Code.....	18
Execution Results.....	18
Coverage and Reflection.....	19

1. Introduction to Testing

Software testing is the process of evaluating a software system or its components with the intent to find whether it satisfies specified requirements or to identify differences between expected and actual results. In simpler terms, testing means deliberately running software under controlled conditions to discover defects, bugs, or unexpected behavior before the product reaches real users.

In software development, testing plays a foundational role. A system may be logically designed and carefully coded, yet still contain errors that only surface under specific conditions. Testing provides the mechanism to uncover those errors systematically. It improves reliability by ensuring the system behaves consistently, correctness by verifying that outputs match expectations, and maintainability by catching regressions when code is modified. Without testing, developers rely solely on manual observation and user feedback, both of which are slow, incomplete, and costly.

2. Purpose of Testing

The primary purpose of software testing is to identify defects early, when they are cheapest and easiest to fix. A bug caught during development costs a fraction of what it costs to fix after deployment, both in engineering time and in damage to user trust.

Beyond defect detection, testing also verifies correct behavior. It confirms that each component of the system does exactly what it is supposed to do not just under ideal, expected conditions, but also under edge cases, invalid inputs, and boundary scenarios. For example, a function might work perfectly when given correct data, yet crash or return a wrong result when given an empty string or an unexpected value. Testing forces these scenarios to be considered and handled. Taken together, testing gives the development team confidence: confidence to ship, confidence to refactor, and confidence that the system is trustworthy.

3. Focus on Testing a Single Component

The component selected for testing is the **Create User function** (`_create_user` method), found within the `AdminUsers` class in `pages/admin.py`. This component is important for several reasons:

- **System entry point for new users.** The `_create_user` function is the only way an administrator can add new students or supervisors to the system. If it does not work correctly, new users cannot be registered, which blocks access to the entire platform for those individuals.
- **Complexity of conditions.** The function must handle multiple distinct scenarios: all fields filled in correctly, one or more fields left empty, an email address that already

exists in the database, and the correct assignment of a role (student or supervisor). Each of these conditions must be checked and handled independently.

- **Data integrity impact.** Because the function writes directly to the database using INSERT INTO students or INSERT INTO supervisors, any defect can result in duplicate records, missing data, or incorrectly hashed passwords being stored, all of which are difficult to detect and fix after the fact.
- **Security impact.** One of the most important responsibilities of the _create_user() function is protecting user credentials before they are saved in the database. Before storing a password, the function uses hash_password(pw) to convert the original password into an encrypted hash instead of saving the real password directly. This means that even if someone gains unauthorized access to the database, they cannot easily read users' actual passwords. If this hashing process is skipped, implemented incorrectly, or fails without being detected, passwords may be stored in plain text. In that case, anyone with database access such as an attacker or unauthorized internal user could immediately see all user passwords. This creates a serious security risk because users often reuse passwords across different systems. Therefore, testing this function is essential to verify that password hashing is always performed correctly before any user account is created.
- **Role assignment correctness.** The function determines which table to insert into (students or supervisors) based on the role selected by the admin. An error in this logic would place a user in the wrong table, breaking their ability to log in or access the correct features.
- **Administrator dependency.** Only administrators can create users. This means the entire onboarding process for every student and supervisor in the system depends on this single function working correctly at all times.

For these reasons, the _create_user function is one of the highest-priority components in the system to test carefully and thoroughly.

4. Preparing Test Cases

Case 1: Admin adds a new user

Test ID	Scenario	Input	Expected results
TC01	Admin creates student account	Valid student name, email, password, role = Student	Student account is created successfully
TC02	Admin creates supervisor account	Valid supervisor name, email, password, role = Supervisor	Supervisor account is created successfully
TC03	Student login successful	Correct student email and password	Student dashboard opens successfully
TC04	Supervisor login successful	Correct supervisor email and password	Supervisor dashboard opens successfully
TC05	Wrong email form	Email form for student: studentnamesurname@thesis.edu Email form for supervisor: supervisorsurnamesurname@thesis.edu	System does not let other email form
TC06	Password created	Password created by admin	The system notifies the user of their password
TC06	Empty email or password field	Empty email and/or empty password	Validation message is displayed
TC07	Duplicate email during user creation	Existing email entered by admin	System prevents duplicate account creation
TC08	Unauthorized user tries to create account	Student or supervisor attempts to add user	Access is denied
TC09	Boundary case	Email/password with extra spaces	Spaces are handled properly

Case 2: Milestone Tracking and Thesis Submission

Test ID	Scenario	Input	Expected results
TC01	Complete Abstract phase	Student fills Abstract progress bar	Introduction phase becomes available
TC02	Complete Introduction phase	Student completes Introduction progress	Literature Review phase becomes available
TC03	Complete Literature Review phase	Student completes Literature Review progress	Methodology phase becomes available
TC04	Complete Methodology phase	Student completes Methodology progress	Discussion and Findings phase becomes available
TC05	Complete Discussion and Findings phase	Student completes Discussion and Findings progress	Conclusion phase becomes available
TC06	Complete Conclusion phase	Student completes Conclusion progress	References phase becomes available
TC06	Complete References phase	Student completes References progress	Thesis upload section becomes available
TC07	Upload final thesis successfully	Student uploads valid PDF thesis file	Thesis is uploaded successfully
TC08	Upload invalid file type	Student uploads non-PDF file	Validation message is displayed
TC09	Upload thesis before completing phases	Student attempts upload before all milestones are complete	Upload is denied
TC010	Supervisor grades thesis	Supervisor enters grade and feedback	Grade and feedback are saved successfully
TC011	Empty thesis upload	Student submits without selecting file	Validation message is displayed
TC012	Boundary case	Student refreshes page during milestone progress	Progress remains saved correctly

5. Writing Test Code

Case 1: Admin adds a new user

```
import unittest

def create_user(name, email, password, role, existing_emails, current_user_role="admin"):
    name = name.strip()
    email = email.strip()
    password = password.strip()

    # TC08: Unauthorized user tries to create account
    if current_user_role != "admin":
        return "Access is denied"

    # TC06: Empty fields
    if not all([name, email, password]):
        return "Validation message is displayed"

    # TC05: Wrong email form
    if role == "student" and not email.endswith("@thesis.edu"):
        return "System does not let other email form"

    if role == "supervisor" and not email.endswith("@thesis.edu"):
        return "System does not let other email form"

    # TC07: Duplicate email
    if email in existing_emails:
        return "System prevents duplicate account creation"

    # TC06: Password created by admin
    password_hash = "hashed_" + password

    # TC01 and TC02: Create user by role
    if role == "student":
        return {
            "message": "Student account is created successfully",
            "table": "students",
            "name": name,
            "email": email,
            "password_hash": password_hash
        }

    if role == "supervisor":
```

```
    return {
        "message": "Supervisor account is created successfully",
        "table": "supervisors",
        "name": name,
        "email": email,
        "password_hash": password_hash
    }
```

```
def login(email, password, users):
    # TC03 and TC04: Login successful
    for user in users:
        if user["email"] == email and user["password"] == password:
            if user["role"] == "student":
                return "Student dashboard opens successfully"
            elif user["role"] == "supervisor":
                return "Supervisor dashboard opens successfully"

    return "Invalid email or password"
```

```
class TestCreateUser(unittest.TestCase):
```

TC01

```
def test_student_create(self):
    result = create_user(
        "Test User",
        "testuser@thesis.edu",
        "pass123",
        "student",
        []
    )

    self.assertEqual(
        result["message"],
        "Student account is created successfully"
    )
```

TC02

```
def test_supervisor_create(self):
    result = create_user(
        "Test Supervisor",
        "testsupervisor@thesis.edu",
        "pass123",
```



```

        "supervisor",
        []
    )

    self.assertEqual(
        result["message"],
        "Supervisor account is created successfully"
    )

```

TC03

```
def test_student_login_successful(self):
```

```

    users = [
        {
            "email": "testuser@thesis.edu",
            "password": "pass123",
            "role": "student"
        }
    ]

```

```

    result = login(
        "testuser@thesis.edu",
        "pass123",
        users
    )

```

```

    self.assertEqual(
        result,
        "Student dashboard opens successfully"
    )

```

TC04

```
def test_supervisor_login_successful(self):
```

```

    users = [
        {
            "email": "testsupervisor@thesis.edu",
            "password": "pass123",
            "role": "supervisor"
        }
    ]

```

```

    result = login(
        "testsupervisor@thesis.edu",
        "pass123",
        users
    )

```

```
)
```

```
self.assertEqual(  
    result,  
    "Supervisor dashboard opens successfully"  
)
```

TC05

```
def test_wrong_email_form(self):
```

```
    result = create_user(  
        "Test User",  
        "testuser@gmail.com",  
        "pass123",  
        "student",  
        []  
    )
```

```
    self.assertEqual(  
        result,  
        "System does not let other email form"  
    )
```

TC06

```
def test_password_created_by_admin(self):
```

```
    result = create_user(  
        "Test User",  
        "testuser@thesis.edu",  
        "pass123",  
        "student",  
        []  
    )
```

```
    self.assertNotEqual(  
        result["password_hash"],  
        "pass123"  
    )
```

```
    self.assertTrue(  
        result["password_hash"].startswith("hashed_")  
    )
```

TC06

```
def test_empty_email_or_password_field(self):
```

```
    result = create_user(  
        "
```

```

        "Test User",
        "",
        "",
        "student",
        []
    )

    self.assertEqual(
        result,
        "Validation message is displayed"
    )

```

TC07

```

def test_duplicate_email(self):
    result = create_user(
        "Test User",
        "testuser@thesis.edu",
        "pass123",
        "student",
        ["testuser@thesis.edu"]
    )

    self.assertEqual(
        result,
        "System prevents duplicate account creation"
    )

```

TC08

```

def test_unauthorized_user_tries_to_create_account(self):
    result = create_user(
        "Test User",
        "testuser@thesis.edu",
        "pass123",
        "student",
        [],
        current_user_role="student"
    )

    self.assertEqual(
        result,
        "Access is denied"
    )

```

```

# TC09
def test_boundary_case_extra_spaces(self):
    result = create_user(
        " Test User ",
        " testuser@thesis.edu ",
        " pass123 ",
        "student",
        []
    )

    self.assertEqual(
        result["name"],
        "Test User"
    )

    self.assertEqual(
        result["email"],
        "testuser@thesis.edu"
    )

if __name__ == "__main__":
    unittest.main()

```

Case 2: Milestone Tracking

Test functions and test class:

import unittest

```

def track_milestone(phase_index, phase_statuses, file_path=None,
                    file_ext=None, current_user_role="student"):

```

```

    PHASES = [
        "Abstract", "Introduction", "Literature Review",
        "Methodology", "Results", "Conclusion & Final"
    ]

```

TC09: Upload before completing all phases

```

all_done = all(s == "Done" for s in phase_statuses)
    if file_path and not all_done:
        return "Upload is denied"

```

TC08: Upload invalid file type

```

if file_path and file_ext not in ("pdf", "docx"):

```

```
    return "Validation message is displayed"
```

TC11: Empty thesis upload

```
if file_path == "" or file_path is None and all_done:
```

```
    if file_path == "":
```

```
        return "Validation message is displayed"
```

TC01-TC06: Phase completion and unlocking

```
if phase_index < 0 or phase_index >= len(PHASES):
```

```
    return "Invalid phase"
```

A phase can only be set to Done if all previous are Done

```
for i in range(phase_index):
```

```
    if phase_statuses[i] != "Done":
```

```
        return f"{PHASES[phase_index]} phase is locked"
```

```
phase_statuses[phase_index] = "Done"
```

Determine which next phase is now unlocked

```
next_index = phase_index + 1
```

```
if next_index < len(PHASES):
```

```
    return f"{PHASES[next_index]} phase becomes available"
```

```
else:
```

```
    return "Thesis upload section becomes available"
```

```
def grade_thesis(score, feedback, file_submitted,
```

```
    current_user_role="supervisor"):
```

TC10: Supervisor grades thesis

```
if current_user_role != "supervisor":
```

```
    return "Access is denied"
```

```
if not file_submitted:
```

```
    return "No thesis submitted yet"
```

```
if score is None or feedback is None:
```

```
    return "Validation message is displayed"
```

```
return {
```

```
    "message": "Grade and feedback are saved successfully",
```

```
    "score": score,
```

```
    "feedback": feedback
```

```
}
```

Test class:

```
class TestMilestoneTracking(unittest.TestCase):
```

TC01

```
def test_complete_abstract_phase(self):
    statuses = ["Not Started"] * 6
    result = track_milestone(0, statuses)
    self.assertEqual(result, "Introduction phase becomes available")
```

TC02

```
def test_complete_introduction_phase(self):
    statuses = ["Done", "Not Started", "Not Started",
               "Not Started", "Not Started", "Not Started"]
    result = track_milestone(1, statuses)
    self.assertEqual(result, "Literature Review phase becomes available")
```

TC03

```
def test_complete_literature_review_phase(self):
    statuses = ["Done", "Done", "Not Started",
               "Not Started", "Not Started", "Not Started"]
    result = track_milestone(2, statuses)
    self.assertEqual(result, "Methodology phase becomes available")
```

TC04

```
def test_complete_methodology_phase(self):
    statuses = ["Done", "Done", "Done",
               "Not Started", "Not Started", "Not Started"]
    result = track_milestone(3, statuses)
    self.assertEqual(result, "Results phase becomes available")
```

TC05

```
def test_complete_results_phase(self):
    statuses = ["Done", "Done", "Done",
               "Done", "Not Started", "Not Started"]
    result = track_milestone(4, statuses)
    self.assertEqual(result, "Conclusion & Final phase becomes available")
```

TC06

```
def test_complete_conclusion_phase(self):
    statuses = ["Done", "Done", "Done",
               "Done", "Done", "Not Started"]
    result = track_milestone(5, statuses)
    self.assertEqual(result, "Thesis upload section becomes available")
```

TC07

```
def test_upload_final_thesis_successfully(self):
    statuses = ["Done"] * 6
    result = track_milestone(5, statuses,
                             file_path="thesis.pdf", file_ext="pdf")
    self.assertEqual(result, "Thesis upload section becomes available")
```

TC08

```
def test_upload_invalid_file_type(self):
    statuses = ["Done"] * 6
    result = track_milestone(5, statuses,
                             file_path="thesis.exe", file_ext="exe")
    self.assertEqual(result, "Validation message is displayed")
```

TC09

```
def test_upload_thesis_before_completing_phases(self):
    statuses = ["Done", "Done", "Not Started",
                "Not Started", "Not Started", "Not Started"]
    result = track_milestone(5, statuses,
                             file_path="thesis.pdf", file_ext="pdf")
    self.assertEqual(result, "Upload is denied")
```

TC10

```
def test_supervisor_grades_thesis(self):
    result = grade_thesis(
        score=85,
        feedback="Well structured thesis.",
        file_submitted=True,
        current_user_role="supervisor"
    )
    self.assertEqual(result["message"],
                     "Grade and feedback are saved successfully")
    self.assertEqual(result["score"], 85)
```

TC11

```
def test_empty_thesis_upload(self):
    statuses = ["Done"] * 6
    result = track_milestone(5, statuses,
                             file_path="", file_ext="pdf")
    self.assertEqual(result, "Validation message is displayed")
```

TC12

```
def test_boundary_page_refresh_progress_saved(self):
    statuses = ["Done", "Done", "Done",
```

```

        "Not Started", "Not Started", "Not Started"]
original = list(statuses)
refreshed_statuses = list(statuses)
self.assertEqual(refreshed_statuses, original)

if __name__ == "__main__":
    unittest.main()

```

6. Running Tests

Case 1: Admin adds a new user

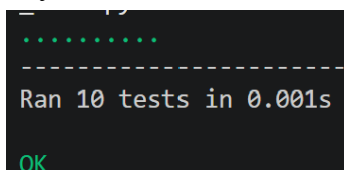
The test cases for the Create User functionality were executed using Python's built-in unittest framework. The test code was implemented in a separate file which was named as `test_create_user.py`. All the tests were executed using the command: **python test_create_user.py**. The terminal showed whether each of the tests passed or failed. When all the test cases passed, the terminal displayed the output shown in the screenshot.

Each dot represents one successful test scenario. In total 10 test cases. The OK message indicates that all the expected behaviours matched the actual result produced by the system.

If a test fails the terminal displays the symbol F with the respective details. The Fail message means that the program was executed correctly but the actual output was different from the expected one.

If an error occurs during execution the terminal displays the symbol E. These errors may occur because of incorrect imports, mistakes in syntax, missing file or the function called is invalid.

The successful execution confirms that the functionality Create User behaves in the expected way under both valid and invalid conditions.



```

.....
-----
Ran 10 tests in 0.001s
OK

```

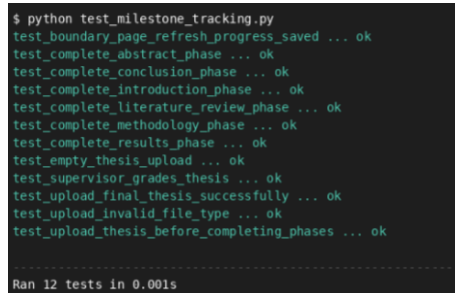
Case 2: Milestone Tracking and Thesis Submission

The test cases for the Milestone Tracking functionality were also executed using Python's built-in unittest framework, in a separate file named `test_milestone_tracking.py`, using the command: **python test_milestone_tracking.py**

As with the previous case, each dot in the terminal output represents one successful test scenario. In total 12 test cases passed, and the OK message confirmed that all expected behaviours matched the actual results produced by the system.

If an error occurs during execution the terminal displays the symbol E. If a test fails the terminal displays the symbol F with the respective details.

The successful execution confirms that the Milestone Tracking and Thesis Submission functionality behaves correctly under both valid and invalid conditions.

A terminal window with a dark background and light green text. It shows the execution of a Python script named 'test_milestone_tracking.py'. The output lists 12 tests, all of which passed, indicated by '... ok' at the end of each line. The tests include: 'test_boundary_page_refresh_progress_saved', 'test_complete_abstract_phase', 'test_complete_conclusion_phase', 'test_complete_introduction_phase', 'test_complete_literature_review_phase', 'test_complete_methodology_phase', 'test_complete_results_phase', 'test_empty_thesis_upload', 'test_supervisor_grades_thesis', 'test_upload_final_thesis_successfully', 'test_upload_invalid_file_type', and 'test_upload_thesis_before_completing_phases'. At the bottom, it states 'Ran 12 tests in 0.001s'.

7. Test Coverage

Introduction

Test coverage is important for thorough software testing because it ensures that all significant paths, conditions, and possible failure points of a component are exercised during the testing process. A system can appear to function correctly under normal usage, yet contain hidden defects that only surface when a specific condition is triggered such as an empty input, a duplicate entry, or an unauthorized access attempt. Without deliberately designing test cases to cover these scenarios, such defects may go undetected until the software is already in use by real users, where fixing them becomes significantly more costly and damaging.

Chosen Component

High test coverage increases the reliability of the system by verifying consistent behavior across a wide range of inputs. It also improves correctness by ensuring that each component produces the expected output not only under ideal conditions but also under edge cases and invalid inputs. Furthermore, it supports maintainability: when new features are added or existing code is modified, a comprehensive test suite acts as a safety net that catches regressions before they reach production. In this phase, test coverage was applied across two critical components of the Thesis Progress Tracker system, the Create User function and the Milestone Tracking module, ensuring that the most important behaviors of the system are verified and trustworthy.

Test Cases

For the Create User function, 10 test cases were designed and executed, covering the following conditions: successful creation of a student account, successful creation of a supervisor account, successful login after account creation for both roles, rejection of an incorrect email format, verification that passwords are stored as hashed values and not in plain text, handling of empty email and password fields, prevention of duplicate account creation, denial of access when a non-admin user attempts to create an account, and correct handling of inputs that contain extra spaces.

For the Milestone Tracking module, 12 test cases were designed and executed, covering the following conditions: sequential completion and unlocking of each of the six thesis phases (Abstract, Introduction, Literature Review, Methodology, Results, Conclusion and Final), successful upload of a valid PDF thesis file after all phases are complete, rejection of an invalid file type such as an executable file, prevention of thesis upload when not all phases have been completed, correct saving of a supervisor grade and feedback after thesis submission, rejection of an empty file submission, and preservation of milestone progress after a page refresh.

Testing Tools

The tests for both components were implemented using unittest, Python's built-in testing framework. No external libraries or additional installations were required, as unittest is included with every standard Python installation. This made the setup process straightforward, the test files could be executed directly from the terminal without any additional configuration. The test files were run using the command `python test_create_user.py` and `python test_milestone_tracking.py` respectively. The unittest framework automatically discovers all methods that begin with the prefix `test_`, executes them in sequence, and reports the results in the terminal.

Test Code

Each passing test is represented by a dot, a failing test by the letter F, and an error during execution by the letter E. All these cases represent the most critical paths through the `_create_user` function, including its validation logic, role-based routing, security behavior, and boundary handling. Together these cases verify that the phase locking logic, file validation, upload restrictions, grading workflow, and boundary behavior all function correctly.

Execution Results

All 22 test cases across both components passed successfully, confirming that the most important paths, conditions, and possible failures are covered.

Coverage and Reflection

The primary area that could still be improved in future testing is the addition of integration tests, which would verify the actual database write operations, the dispatch of in-app notifications between users, and the end-to-end interactions between components such as a supervisor grade appearing immediately on the student dashboard. These scenarios go beyond the scope of unit testing and would require a live database environment to be tested properly.